

CSE312 HW1
PART-A REPORT

MUHAMMET FURKAN YILGIN
220104004964

1. Algorithms

First of all I would like to start from the simplest algorithms which includes linear search, binary search, collatz sequence and long running program. The photos of the implementations are below.

```
int LongRunningProgram(int number){
    int result = 0;
    for(int i=0; i<number ; i++){
        for(int j=0; j<number ; j++){
            result += i * j ;
        }
    }
    return result;
}

void LongRunningProgramTesting(){
    printf("--- LONG RUNNING PROGRAM ---\n");
    printf("Result of 100 is -> ");
    int result = LongRunningProgram(100);
    printDigit(result);
    printf("\n");
    sleep(5);
    SyscallHandler::sys_exit();
}
```

```
int LinearSearch(int * array, int size, int target)
{
    for (int i = 0; i < size; i++){
        if (array[i] == target){
            return i;
        }
    }
    return -1;
}

void LinearSearchTesting() {
    int array[] = {10, 20, 80, 30, 60, 50, 110, 100, 130, 170};
    int size = sizeof(array) / sizeof(int); // getting the size
    int target;
    printf(" --- LINEAR SEARCH ---\n");
    printf("OUR ARRAY IS -> "); printArr(array, size);
    scanfPrompt("VALUE TO BE FOUND IS -> ", &target);
    int res = LinearSearch(array, size, target); // getting the result of the search
    printf("LINEAR-SEARCH(");
    printDigit(target);
    printf("): ");
    printDigit(res);
    printf("\n");
    sleep(5);
    SyscallHandler::sys_exit();
}
```

```

int BinarySearch(int *arr, int target, int low, int high){
    if (low > high){
        return -1;
    }
    int mid = (high + low) / 2;
    if (arr[mid] == target){
        return mid;
    }
    else if (target < arr[mid]){
        return BinarySearch(arr, target, low, mid - 1);
    }
    else{
        return BinarySearch(arr, target, mid + 1, high);
    }
}

void BinarySearchTesting(){
    int array[] = {10, 20, 80, 30, 60, 50, 110, 100, 130, 170};
    int size = sizeof(array) / sizeof(int);
    int target;
    printf("--- BINARY SEARCH ---\n");
    printf("OUR ARRAY IS -> "); printArr(array, size);
    // first sort and print the array
    SortForBinary(array, size);
    printf("SORTING THE ARRAY...\nSORTED ->"); printArr(array, size);
    scanfPrompt("OUR TARGET IS -> ", &target);
    int res = BinarySearch(array, target, 0, size - 1);
    printf("Result of binary search for target ");
    printDigit(target);
    printf(" is -> ");
    printDigit(res);
    printf("\n");
}

```

```

// Function to generate the Collatz sequence for a given number n.
// The sequence is stored in the provided buffer `buffer`.
void CollatzSequence(int n, int *buffer) {
    int i = 0; // Initialize index to store the sequence
    buffer[0] = n; // Store the initial number in the buffer

    // Continue generating the sequence until n becomes 1
    while (n > 1) { // to ensure we don't exceed buffer size
        if (n % 2 == 0) {
            n = n / 2; // If n is even, divide it by 2
        } else {
            n = (3 * n) + 1; // If n is odd, multiply by 3 and add 1
        }
        buffer[++i] = n; // Store the new value in the buffer
    }
}

void CollatzSequenceTesting() {
    int str[64];
    int buff[256];
    int n;
    printf(" --- COLLATZ SEQUENCE --- \n");
    scanfPrompt("Current Sequence is -> ", &n);
    printf("Collatz("); printDigit(n); printf("):\n");
    for (int i = n; i > 0; --i) {
        CollatzSequence(i, buff);
        printSeq(buff);
    }
    SyscallHandler::sys_exit();
}

```

2. Lifecycle of PART-A

I would like to show the implementation and outputs of the lifecycle wanted in PART-A which is loading each program 3 times.

Implementation :

```
void lifecycleA(GlobalDescriptorTable *gdt, TaskManager *tm){
    // Loading each of the 2 program(collatz and long running) 3 times.
    Task taskCollatz(gdt, CollatzSequenceTesting, 2, Normal);
    Task taskLongRunningProgram(gdt, LongRunningProgramTesting, 1, Normal);
    for (int i = 0; i < 3; i++){
        tm->AddTask(&taskCollatz);
        tm->AddTask(&taskLongRunningProgram);
    }
}
```

Outputs :

```
Welcome to FurkanOS
LIFECYCLE OF PART A
--- COLLATZ SEQUENCE ---
Current Sequence is -> 7
Collatz(7):
7: 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
6: 3 10 5 16 8 4 2 1
5: 16 8 4 2 1
4: 2 1
3: 10 5 16 8 4 2 1
2: 1
1: 1
--- EXIT SYSCALL IS CALLED ---
--- LONG RUNNING PROGRAM ---
Result of 100 is -> 24502500
--- EXIT SYSCALL IS CALLED ---
--- COLLATZ SEQUENCE ---
Current Sequence is -> 1
Collatz(1):
1: 1
--- EXIT SYSCALL IS CALLED ---
--- LONG RUNNING PROGRAM ---
Result of 100 is -> 24502500
--- COLLATZ SEQUENCE ---
Current Sequence is -> 5
Collatz(5):
5: 16 8 4 2 1
4: 2 1
3: 10 5 16 8 4 2 1
2: 1
1: 1
--- EXIT SYSCALL IS CALLED ---
--- LONG RUNNING PROGRAM ---
Result of 100 is -> 24502500
--- EXIT SYSCALL IS CALLED ---
--- EXIT SYSCALL IS CALLED ---
```

3. Syscalls

Now I would like to show the implementations of the syscalls. The photos of the implementations of the syscalls are below with the comments explaining how they were implemented.

Exit :

```
void TaskManager::Exit() {
    // Print a message indicating that the exit system call has been invoked.
    printf("--- EXIT SYSCALL IS CALLED ---\n");

    // Retrieve the current task using the currentTask index.
    Task *current = &tasks[currentTask];

    // Checking if the current task's PID (Process ID) is 1, which typically indicates
    // the init process. The init process is important and it should
    // not be terminated. If the current task is the init process, just returning
    // without performing any further actions.
    if (current->pid == 1)
        return;

    // Check if the current task's parent is waiting for this task to terminate.
    if (current->waitparent) {
        // Find the index of the parent task using the parent's PID (ppid).
        int index_of_parent = FindTask(current->ppid);
        // If the parent task is found (index is valid), set the parent's state to Ready.
        // This indicates that the parent can continue execution after the child terminates.
        if (index_of_parent >= 0)
            tasks[index_of_parent].state = Ready;
    }

    // Decrement the active task counter as this task is about to be terminated.
    --Active_Task_counter;

    // Set the current task's state to Terminated to mark it as no longer active.
    current->state = TaskState::Terminated;
}
```

Execve :

```
common::uint32_t TaskManager::Execve(void (*entrypoint)()) {
    // Printing execve system call is called.
    printf("--- EXECVE SYSCALL IS CALLED ---\n");

    // Get a pointer to the current task
    Task *current = &tasks[currentTask];

    // Reinitialize the current task with the new entry point
    // Keep the same PID, PPID and priority
    current->InitTask(gdt, entrypoint, current->pid, current->ppid, current->priority);

    // Return the CPU state of the current task which will now start executing the new entry point
    return (common::uint32_t) current->cpustate;
}
```

Waitpid :

```
34 void TaskManager::Waitpid(common::uint32_t pid) {
35     // Printing waitpid system call is called.
36     printf("--- WAITPID SYSCALL IS CALLED ---\n");
37
38     // Get a pointer to the current task (the parent)
39     Task *parent = &tasks[currentTask];
40
41     // If the pid is greater than 0, wait for a specific child process to finish
42     if (pid > 0) {
43         // Find the index of the task with the given pid
44         int childIndex = FindTask(pid);
45         // Make sure there is such a child process
46         if (childIndex != -1 && tasks[childIndex].ppid == parent->pid) {
47             // Set the parent task state to Blocked
48             parent->state = Blocked;
49             // Set the waitparent flag of the child task to true
50             // The parent will wake up when the child task is terminated
51             tasks[childIndex].waitparent = true;
52         }
53     }
54     else {
55         // If pid is -1, set the waitparent flag to true for all child processes
56         for (int i = 0; i < Task_counter; ++i) {
57             // Check if the task is a child of the parent
58             if (tasks[i].ppid == parent->pid) {
59                 // Set the parent task state to Blocked
60                 parent->state = Blocked;
61                 // Set the waitparent flag of the child task to true
62                 tasks[i].waitparent = true;
63             }
64         }
65     }
66 }
67
```

Fork :

```
common::uint32_t TaskManager::Fork(CPUState *cpustate) {
    printf("--- FORK SYSCALL IS CALLED ---\n"); // message to state fork is called.

    if (Task_counter >= 256) // checking if the max num of tasks reached
        return -1; // then returning -1 to indicate.

    // Get pointers to the parent task (current task) and the new child task
    Task *parent = &tasks[currentTask];
    Task *child = &tasks[Task_counter];

    // Initializing the child task with the parent's entry point, next available PID,
    // parent's PID, and the same priority as the parent
    child->InitTask(gdt, (void(*)()) parent->cpustate->eip, next_pid, parent->pid, parent->priority);

    // Copying the CPU state from the parent to the child
    *(child->cpustate) = *cpustate;

    IncrementPIDS_TASKS(); // Increment the next available PID, the task counter, and the active task counter
    return child->pid; // Returning the PID of the new created child task
}
```

Implementation of the Other Required Functions

```
60 int32_t rand(int min, int max) {
61     uint64_t counter;
62     int32_t num;
63
64     // Read the clock counter using the rdtsc instruction
65     asm("rdtsc" : "=A"(counter));
66
67     // Use the clock counter as a source of randomness
68     // Apply a linear congruential generator (LCG) formula
69     counter = counter * 1103515245 + 12345;
70
71     // Scale down the large number to the desired range 0 to max - min
72     num = (int)(counter / 65536) % (max - min);
73
74     // Ensure the number is non-negative
75     if (num < 0)
76         num += max;
77
78     // Adjust the number to the range min to max
79     return num + min;
80 }
81
82
83 void sleep(int seconds) {
84     long int delay = 100000000; // Define the delay for 1 second
85
86     // Loop to create a delay equivalent to the number of seconds
87     for (long int i = 0; i < seconds * delay; ++i)
88         i++; // Increment to create a delay (NOP-like operation)
89 }
90
```

```
int scanf(char *out) {
    char *str;
    int i;

    // Lock I/O operations to prevent interrupts during input
    InterruptManager::setIOLock(true);

    // Read the whole line until a newline character is encountered
    while ((str = InterruptManager::stdin.read('\n')) == nullptr);

    // Copy the input string to the output buffer until a newline or null character
    for (i = 0; str[i] != '\0' && str[i] != '\n'; ++i)
        out[i] = str[i];

    // Null-terminate the output string
    out[i] = '\0';

    // Unlock I/O operations
    InterruptManager::setIOLock(false);

    // Return the number of characters read
    return i;
}
```

Example Process Tables Taken from Lifecycle Outputs

```
12      1      NORMAL PRIORITY    READY
13      1      NORMAL PRIORITY    TERMINATED
14      1      NORMAL PRIORITY    READY
15      1      NORMAL PRIORITY    TERMINATED
16      1      NORMAL PRIORITY    READY
17      1      NORMAL PRIORITY    TERMINATED
18      1      NORMAL PRIORITY    TERMINATED
19      1      NORMAL PRIORITY    READY
20      1      NORMAL PRIORITY    TERMINATED
21      1      NORMAL PRIORITY    TERMINATED
22      1      NORMAL PRIORITY    TERMINATED
23      1      NORMAL PRIORITY    READY
24      1      NORMAL PRIORITY    TERMINATED
25      1      NORMAL PRIORITY    TERMINATED
26      1      NORMAL PRIORITY    RUNNING
27      1      NORMAL PRIORITY    READY
28      1      NORMAL PRIORITY    READY
29      1      NORMAL PRIORITY    READY
*****
--- COLLATZ SEQUENCE ---
Current Sequence is ->
```

```
8       1      NORMAL PRIORITY    TERMINATED
9       1      NORMAL PRIORITY    TERMINATED
10      1      NORMAL PRIORITY    TERMINATED
11      1      NORMAL PRIORITY    TERMINATED
12      1      NORMAL PRIORITY    TERMINATED
13      1      NORMAL PRIORITY    TERMINATED
14      1      NORMAL PRIORITY    TERMINATED
15      1      NORMAL PRIORITY    TERMINATED
16      1      NORMAL PRIORITY    TERMINATED
17      1      NORMAL PRIORITY    TERMINATED
18      1      NORMAL PRIORITY    TERMINATED
19      1      NORMAL PRIORITY    RUNNING
20      1      NORMAL PRIORITY    TERMINATED
21      1      NORMAL PRIORITY    TERMINATED
22      1      NORMAL PRIORITY    TERMINATED
23      1      NORMAL PRIORITY    TERMINATED
24      1      NORMAL PRIORITY    TERMINATED
25      1      NORMAL PRIORITY    TERMINATED
26      1      NORMAL PRIORITY    TERMINATED
27      1      NORMAL PRIORITY    TERMINATED
28      1      NORMAL PRIORITY    TERMINATED
29      1      NORMAL PRIORITY    TERMINATED
*****
--- EXIT SYSCALL IS CALLED ---
```

I didn't use the ForkTask function you have provided since I wasn't able to make it work for my code so instead of not doing it entirely I have done it this way.

Rest of the implementations are shown in Report of PART-B which includes scheduling algorithms, adding tasks and lifecycles.